TECHNOLOGY

AWS Solution Architect

©Simplilearn. All rights reserved.

simplilearn

Monitoring and Automation

©Simplilearn. All rights reserved.

Learning Objectives

By the end of this lesson, you will be able to:

- Monitor instances effectively by configuring and utilizing CloudWatch alarms
- Create CloudFormation templates to automate infrastructure provisioning and management
- Update resources within an existing infrastructure using CloudFormation, ensuring scalability and maintainability
- Build complete infrastructures efficiently using CloudFormation, streamlining the deployment process for AWS resources

©Simplilearn. All rights reserved.

CloudWatch

©Simplilearn. All rights reserved.

CloudWatch: Overview

CloudWatch is a service that monitors resources and applications run by users on AWS.

Amazon CloudWatch

- It enables users to collect and access performance and operational data in the form of logs and metrics.
- It provides comprehensive visibility into resource utilization, application performance, and operational health.

©Simplilearn. All rights reserved.

CloudWatch: Features

- It sets alarms and initiates automated actions.
- You can integrate CloundWatch with notification services.
- It monitors applications, infrastructure, and services.
- It helps in the collection and tracking of predefined and custom metrics.
- It creates dashboards to display metrics.

©Simplilearn. All rights reserved.

CloudWatch: Workflow

Here are the tasks to perform in the CloudWatch workflow:

1. **Control:** Collect metrics and logs from AWS services running on AWS and on-premises servers.
2. **Monitor:** Provide a uniform operational view through dashboards.
3. **Act:** Automate responses to operational changes using events and autoscaling with CloudWatch.
4. **Analyze:** Conduct real-time analysis using CloudWatch Metric Math.
5. **Compliance and Security:** Integrate with IAM to track user activities on resources.

©Simplilearn. All rights reserved.

CloudWatch: Metrics

CloudWatch Metrics offers performance metrics with two options: basic, which is free, and detailed, which is a paid service.

It maintains metric data for up to 15 months, enabling users to establish alarms based on specific metric values.

Users can also utilize CloudWatch Metrics for searching and displaying data on dashboards.

©Simplilearn. All rights reserved.

CloudWatch: Metrics

- A timestamp can be set up to two weeks in the past or two hours in the future.
- If the user does not provide a timestamp, CloudWatch will generate one based on the time it received the data point.

©Simplilearn. All rights reserved.

CloudWatch: Metrics

- Timestamps in CloudWatch consist of DateTime objects, including the full date, hours, minutes, and seconds.
- It is advisable to use Coordinated Universal Time (UTC) for time-related operations in CloudWatch.
- CloudWatch alarms evaluate metrics based on the current UTC.
- Using custom metrics with non-current UTC timestamps may lead to insufficient data states or delayed alarms.

©Simplilearn. All rights reserved.

CloudWatch: Metrics

CloudWatch retains metric data as follows:

- Data points with a period of fewer than 60 seconds are available for 3 hours.
- Data points with a period of 60 seconds are available for 15 days.
- Data points with a period of 300 seconds are available for 63 days.
- Data points with a period of 3600 seconds are available for 455 days (15 months).

©Simplilearn. All rights reserved.

CloudWatch: Metrics

CloudWatch aggregates data points initially published within a shorter period for long-term storage.

Example:

- If the user collects data at a one-minute interval, the data stays available for 15 days at a one-minute resolution.
- After 15 days, CloudWatch aggregates the data, making it retrievable at a five-minute resolution.
- After 63 days, CloudWatch further aggregates the data, making it available at a one-hour resolution.

©Simplilearn. All rights reserved.

CloudWatch: Alarms

Users employ an alarm to automatically initiate actions.

- An alarm monitors a single metric over a specified time, taking actions based on the metric's value in comparison to a threshold.
- The alarm issues an alert to an Auto Scaling policy or an Amazon SNS subject as the corresponding action.1

©Simplilearn. All rights reserved.

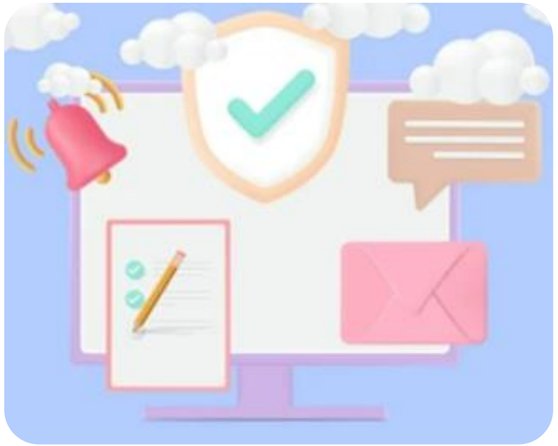
CloudWatch: Alarms

- The alarm invokes action only if the state change is sustained.
- The CloudWatch alarms do not invoke actions while in a particular state.
- A state change must be maintained for a specified number of periods before actions are invoked.

©Simplilearn. All rights reserved.

CloudWatch: Alarms

When setting up an alarm, a user must select a monitoring period equal to or greater than the metric's resolution.



High-resolution alarms incur higher charges than standard-resolution alarms.

©Simplilearn. All rights reserved.

CloudWatch: Alarms

An example:

- Amazon EC2's basic monitoring service provides metrics for an instance every five minutes.
- When setting an alarm on a basic monitoring metric, a user must select a specific duration.
- For high-resolution metrics, a user may choose a standard alarm with a period that is a multiple of 60 seconds.

©Simplilearn. All rights reserved.

CloudWatch: Logs

Users can utilize Amazon CloudWatch Logs to monitor, store, and access log files from sources such as Amazon EC2 instances, AWS CloudTrail, and Route 53.

©Simplilearn. All rights reserved.

CloudWatch: Logs

CloudWatch Logs enables users to centralize logs from all their systems, applications, and AWS services into a single, scalable service.

Once centralized, users can view these logs, search for specific error codes, filter by fields, or securely archive them for future analysis.

©Simplilearn. All rights reserved.

CloudWatch: Logs

The features of CloudWatch Logs include:

- **Alarms:** These notify API activities, aiding effective troubleshooting for the architect.
- **Retention Policies:** They help users to adjust the retention duration from one day to 10 years.
- **Route 53 DNS Queries Logging:** It allows the recording of DNS requests received by Route 53 resolver.

©Simplilearn. All rights reserved.

Automating EC2 Instance Shutdown with CloudWatch Alarm

Duration:15 min

Problem Statement:

You have been assigned a task to create a CloudWatch alarm to automatically stop an instance based on CPU utilization.

Outcome:

By performing this demo, you will learn how to create a CloudWatch alarm to automatically stop an instance based on CPU utilization. This will enhance your ability to automate resource management and optimize cost efficiency on AWS.

Note: Refer to the demo document for detailed steps:
01_Automating_EC2_Instance_Shutdown_with_CloudWatch_Alarm

ASSISTED PRACTICE

©Simplilearn. All rights reserved.

Assisted Practice: Guidelines

Steps to be followed are:

1. Create a CloudWatch alarm

©Simplilearn. All rights reserved.

Evaluating CloudWatch Log Reports

Duration:10 min

Problem Statement:

You have been assigned a task to evaluate CloudWatch Log reports by creating and graphing metrics from log data using CloudWatch Log groups and Metric filters for monitoring and analysis.

Outcome:

By performing this demo, you will learn how to evaluate CloudWatch Log reports by creating and graphing metrics from log data using CloudWatch Log Groups and Metric Filters. This will enhance your ability to monitor and analyze application performance and operational health on AWS.

Note: Refer to the demo document for detailed steps:
02_Evaluating_CloudWatch_Log_Reports

ASSISTED PRACTICE

©Simplilearn. All rights reserved.

Assisted Practice: Guidelines

Steps to be followed are:

1. Evaluate CloudWatch Log reports

©Simplilearn. All rights reserved.

Quick Check

An AWS administrator needs to automatically shut down an EC2 instance when its CPU utilization remains below 10% for a sustained period. Which combination of features should they configure in CloudWatch to achieve this?

A. CloudWatch Logs and CloudWatch Metrics

B. CloudWatch Metrics and CloudWatch Alarm

C. CloudWatch Alarms and CloudWatch Logs

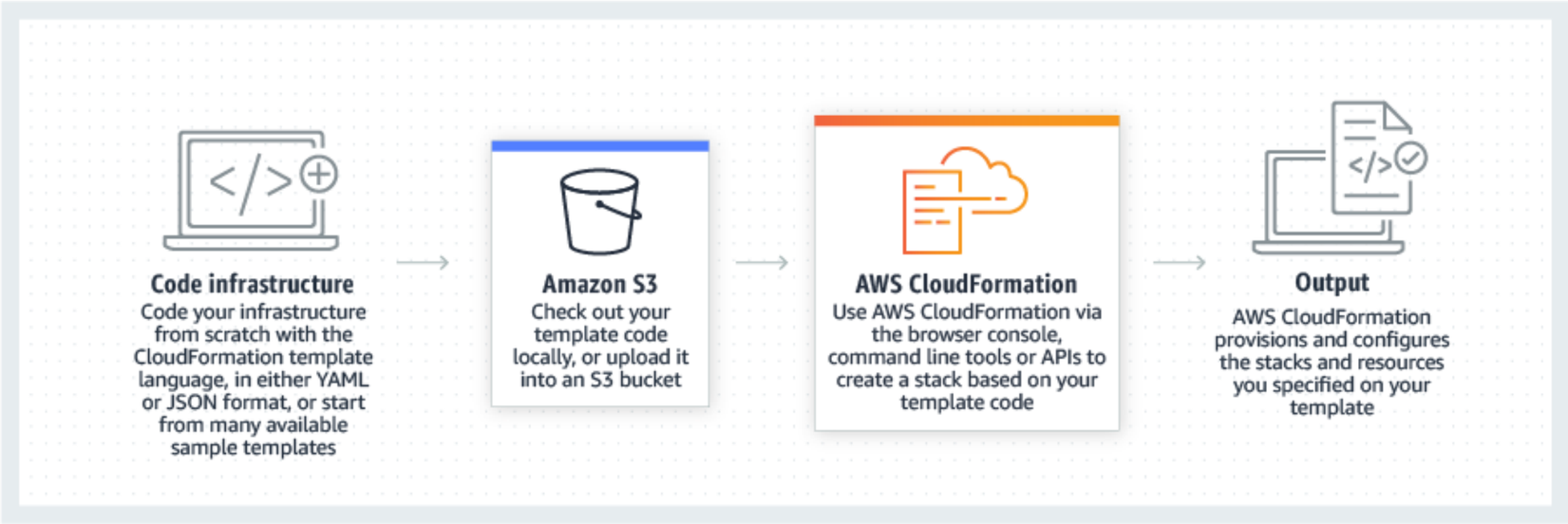
D. CloudWatch Logs and CloudWatch Events

CloudFormation

©Simplilearn. All rights reserved.

Introduction to CloudFormation

AWS CloudFormation is a service that helps an end-user automate AWS services by creating templates.



The CloudFormation interprets the template and makes the API calls to create the resources.

Image source: https://aws.amazon.com/cloudformation/

©Simplilearn. All rights reserved.

Introduction to CloudFormation

©Simplilearn. All rights reserved.

Introduction to CloudFormation

Templates and **Stacks** are important elements of CloudFormation.

A **template** is a JSON or YAML-formatted text file.

A **stack** is a collection of AWS resources that a user can manage as a single unit.

©Simplilearn. All rights reserved.

Types of Templates

Template in JSON format:

```
{
  "AWSTemplateFormatVersion" : "version date",

"Description" : "JSON string",
"Metadata" : {
    template metadata },
"Parameters" : {
    set of parameters },
  "Rules" : {
    set of rules },
"Mappings" : {
    set of mappings },
"Conditions" : {
    set of conditions },
"Transform" : {
    set of transforms},
"Resources" : {
    set of resources  },
 "Outputs" : {
    set of outputs  }
}
```

©Simplilearn. All rights reserved.

Types of Templates

Template in YAML format:

```
AWSTemplateFormatVersion: "version date"
Description:
  String
Metadata:
  template metadata
Parameters:
  set of parameters
Rules:
  set of rules
Mappings:
  set of mappings
Conditions:
  set of conditions
Transform:
  set of transforms
Resources:
  set of resources
Outputs:
  set of outputs
```

©Simplilearn. All rights reserved.

Template Structure: Overview

The sections of the template are as follows:

01

Data tables (optional):
This section includes static configuration values, such as AMI names.

02

Outputs (optional):
This section includes values that are returned whenever users view their stack's properties.

03

Parameters (optional):
This section includes input values that are provided while creating stacks.

04

Resources (required):
This section includes the names and configurations of services to be added.

©Simplilearn. All rights reserved.

Template Structure: Overview

The sections of the template are as follows:

05

Conditions (optional)

This section includes actions to be taken based on conditional statements, such as equals.

06

Mappings (optional)

This section has set values based on a region, such as AMIs.

07

Transform (optional)

This section includes snippets from outside the main template.

©Simplilearn. All rights reserved.

Template Structure: Overview

Template in JSON format

```
{"AWSTemplateFormatVersion" : "version date",

"Description" : "JSON string",
"Metadata" : {
   template metadata },
"Parameters" : {
   set of parameters },
 "Rules" : {
   set of rules },
"Mappings" : {
   set of mappings },
"Conditions" : {
   set of conditions },
"Transform" : {
   set of transforms},
"Resources" : {
   set of resources  },
 "Outputs" : {
   set of outputs  }}
```

The template contains the following sections:

- Template version
- Description
- Metadata
- Parameter
- Mapping
- Conditions
- Output
- Resources

©Simplilearn. All rights reserved.

Parameter Section: Example

Here is an example to show how to specify input parameters for a template:

```
"Parameters“ : {

 "InstanceTypeParameter":  {

"Type“ :  "String",

 "Default“  : "t2.micro",

"AllowedValues" : ["t2.micro", "m1.small",“ml.large"],

"Description“ : "Enter t2.micro, m1.small, or m1.large. Default is t2.micro.“
  }
}
```

©Simplilearn. All rights reserved.

Mapping Section: Example

Here is an example to show how to match a key to a corresponding set of named values:

```
"Mappings": {
   "Mapping01“ : {
      "Key01“ : {
        "Name“:"Value01“
       }

     "Key02": {
        "Name“ : "Value02“
        }
     "Key03": {
          "Name“ : "Value03“
        }
    }
}
```

©Simplilearn. All rights reserved.

Condition Section

The properties of the condition section of the template are:

01

The condition statement is optional.

02

It contains statements that define the circumstances under which the resources are created.

©Simplilearn. All rights reserved.

Resources Section: Example

Here is an example to show how to declare the AWS resources that need to be included in the stack:

```
"Resources": {
    "SQSQ44CP0": {
      "Type": "AWS::SQS::Queue",
      "Properties": {},
      "Metadata": {
        "AWS::CloudFormation::Designer": {
          "id": "3f3e280c-2e3b-43d2-bdaa-9c9a529f6de8"
        }
      }
    },
    "S3B4XB7F": {
      "Type": "AWS::S3::Bucket",
      "Properties": {},
      "Metadata": {
        "AWS::CloudFormation::Designer": {
          "id": "e5cbfee9-294b-4688-afd2-43274d15a8bc"
```

©Simplilearn. All rights reserved.

Creating an S3 Bucket Using CloudFormation

Duration:10 min

Problem Statement:

You have been assigned a task to create an S3 Bucket stack using CloudFormation for efficient infrastructure management and deployment.

Outcome:

By performing this demo, you will learn how to create an S3 bucket stack using CloudFormation. This will enhance your skills in infrastructure as code (IaC) and enable you to efficiently manage and automate AWS resources.

Note: Refer to the demo document for detailed steps:
03_Creating_an_S3 Bucket_Using_CloudFormation

ASSISTED PRACTICE

©Simplilearn. All rights reserved.

Assisted Practice: Guidelines

Steps to be followed are:

1. Create a template
2. Create an IAM role for the S3 bucket stack
3. Create an S3 Bucket stack

©Simplilearn. All rights reserved.

Deleting an Existing Stack

Duration:10 min

Problem Statement:

You have been assigned a task to delete an existing S3 bucket stack for resource management and cost optimization.

Outcome:

By performing this demo, you will learn how to delete an existing stack, focusing on deleting an S3 bucket. This will enhance your skills in managing AWS resources and ensure you understand the cleanup process involved in infrastructure management.

Note: Refer to the demo document for detailed steps:
04_Deleting_an_Existing_Stack

ASSISTED PRACTICE

©Simplilearn. All rights reserved.

Assisted Practice: Guidelines

Steps to be followed are:

1. Locate an existing stack and delete it

©Simplilearn. All rights reserved.

Updating an Existing Stack

Duration:10 min

Problem Statement:

You have been assigned a task to update an existing stack of an S3 bucket for maintaining and modifying infrastructure.

Outcome:

By performing this demo, you will learn how to update an existing stack, focusing on deleting an S3 bucket. This will enhance your skills in managing and modifying AWS resources using CloudFormation, ensuring efficient and controlled updates to your infrastructure.

Note: Refer to the demo document for detailed steps:
05_Updating_an_Existing_Stack

ASSISTED PRACTICE

©Simplilearn. All rights reserved.

Assisted Practice: Guidelines

Steps to be followed are:

1. Create an S3 bucket stack using CloudFormation Designer
2. Update an S3 bucket from the stack

©Simplilearn. All rights reserved.

Updating Input Parameters in an Existing Stack

Duration:10 min

Problem Statement:

You have been assigned a task to update input parameters in an existing CloudFormation stack for flexible and dynamic infrastructure management.

Outcome:

By performing this demo, you will learn how to update input parameters in an existing CloudFormation stack. This will enhance your skills in managing and customizing AWS resources, ensuring flexible and efficient infrastructure updates.

Note: Refer to the demo document for detailed steps:
06_Updating_Input_Parameters_in_an_Existing_Stack

ASSISTED PRACTICE

©Simplilearn. All rights reserved.

Assisted Practice: Guidelines

Steps to be followed are:

1. Update the template and test the updates

©Simplilearn. All rights reserved.

Quick Check

A developer wants to update an existing CloudFormation stack to add a new S3 bucket without disrupting the current resources. Which approach should they take?

A. Delete the existing stack and create a new one with the S3 bucket

B. Manually create the S3 bucket in the AWS Management Console

C. Update the CloudFormation template and use the **Update Stack** option

D. Create a new stack for the S3 bucket and link it to the existing stack

Key Takeaways

- CloudWatch is a monitoring and observational service used to monitor resources and applications that a user runs on AWS.
- AWS CloudFormation is a service that helps to model and set up AWS resources in an efficient manner.
- Templates and stacks are the major elements of CloudFormation.

©Simplilearn. All rights reserved.

Creating an Alarm Using CloudWatch

Duration: 30 Mins

Project agenda: To create an alarm using CloudWatch that will allow you to watch CloudWatch metrics (CPU utilization) with a given threshold and receive notifications when the metrics fall outside the threshold levels that you configure.

Description: Launch 3 virtual machine instances (Linux), perform tasks on these VMS of your choice, and set up a dashboard with metrics showing CPU utilization of all 3 VMS.

Perform the following:

1. Launch Linux VMs
2. Connect SSH to VMs
3. Perform Linux-related tasks on the VM
4. Configure the CloudWatch services
5. Create metrics for CPU utilization for all VMs
6. Create an alarm and send a notification through SNS

LESSON-END PROJECT

©Simplilearn. All rights reserved.

Thank You

©Simplilearn. All rights reserved.